

Actividad de Sistemas Distribuidos

Aplicacion cliente-servidor con RMI:
desarrollo con la libreria LipeRMI y con RMI estandar de Java

Dato	Informacion
Trabajo	Actividad academica individual
Autor	Caled Vivanco
Docente y autor de la guia	John Carlos Arrieta Arrieta
Fecha	27 de septiembre de 2026
Tema	Aplicaciones distribuidas con RMI y RPC

Contenido de la entrega. El documento responde los tres puntos de la actividad academica: (1) la documentacion paso a paso del desarrollo del ejemplo de la guia con la libreria LipeRMI, junto con los fuentes de los tres proyectos; (2) el mismo ejemplo resuelto con RMI estandar de Java, con la explicacion de las diferencias y similitudes frente a LipeRMI; y (3) la investigacion sobre otras librerias para construir sistemas distribuidos con RMI/RPC y un ejemplo muy simple desarrollado con una de ellas.

Contenido

1. Introduccion	3
1.1 Objetivo de la actividad	3
1.2 Organizacion de la entrega	3
2. Punto 1: desarrollo paso a paso con LipeRMI	4
2.1 Paso 0: herramientas y libreria	4
2.2 Estructura de los tres proyectos	4
2.3 Paso 1: proyecto tipo libreria	4
2.4 Paso 2: proyecto servidor	5
2.5 Paso 3: proyecto cliente	6
2.6 Paso 4: ejecucion y pruebas	6
3. Punto 2: el mismo ejemplo con RMI estandar de Java	8
3.1 Paso 1: proyecto libreria	8
3.2 Paso 2: proyecto servidor	8
3.3 Paso 3: proyecto cliente	8
3.4 Paso 4: ejecucion y pruebas	9
3.5 Diferencias y similitudes entre RMI estandar y LipeRMI	9
4. Punto 3: otras librerias para RMI/RPC	11
4.1 Librerias encontradas	11
4.2 Analisis de la informacion	11
4.3 Ejemplo muy simple desarrollado con gRPC	11
5. Resultados y verificacion	14
5.1 Casos de prueba utilizados en las dos tecnologias RMI	14
5.2 Como reproducir la verificacion	14
6. Conclusiones	15
7. Referencias	16

1. Introduccion

Una aplicacion distribuida es un programa cuyas partes se ejecutan en computadores diferentes y se comunican por la red. Para construirla sin programar directamente los sockets existen tecnologias de invocacion remota de metodos (RMI) y de llamada a procedimientos remotos (RPC), cuyo objetivo es que el programador escriba una llamada como si el metodo estuviera en su propio programa, aunque en realidad se ejecute en otra maquina.

El ejemplo desarrollado en toda la actividad es el de la guia del docente: una calculadora de Indice de Masa Corporal (IMC). El usuario ingresa el peso y la altura en un formulario; el cliente envia esos datos por la red y el **calcula se ejecuta en el servidor**, que devuelve el resultado y su interpretacion. La formula aplicada es $IMC = peso / (altura * altura)$ y las categorias son las mismas de la guia: bajo peso, peso normal, sobrepeso y obesidad.

1.1 Objetivo de la actividad

- Documentar cada paso del desarrollo del ejemplo de la guia usando la libreria de terceros LipeRMI, entregando los fuentes de los tres proyectos: libreria, servidor y cliente.
- Repetir el mismo ejemplo con RMI estandar de Java (paquete java.rmi), sin LipeRMI, documentando el proceso y explicando las diferencias y similitudes entre las dos tecnologias.
- Investigar otras librerias, en Java y en otros lenguajes, que permitan desarrollar sistemas distribuidos con RMI/RPC, y entregar un ejemplo muy simple desarrollado con una de ellas.

1.2 Organizacion de la entrega

Los fuentes se entregan en tres carpetas, una por punto de la actividad. Cada carpeta contiene los proyectos y su documento de soporte con el paso a paso, de modo que cualquier persona pueda compilar, ejecutar y verificar el ejemplo.

Punto	Carpeta	Contenido
1	1-lipermi	Tres proyectos Maven: EjemploLipeRmiLibImc, EjemploLipeRmiServidorImc y EjemploLipeRmiClienteImc, mas la documentacion paso a paso.
2	2-rmi-estandar	Los mismos tres proyectos contruidos con java.rmi, mas la documentacion del proceso y la comparacion con LipeRMI.
3	3-investigacion	Documento de investigacion sobre librerias RMI/RPC y el proyecto ejemplo-grpc-java, un ejemplo minimo y funcional.

2. Punto 1: desarrollo paso a paso con LipeRMI

LipeRMI es una librería de terceros que implementa el mecanismo de invocación remota de Java de una forma más sencilla que el paquete estándar del JDK: no exige que la interfaz remota herede de `java.rmi.Remote`, no obliga a declarar `RemoteException` en cada método y no necesita el registro `rmiregistry`, porque publica el servicio en un solo puerto TCP.

2.1 Paso 0: herramientas y librería

Herramienta	Uso en el desarrollo
JDK 17	Compilar y ejecutar los tres proyectos.
IDE (NetBeans o IntelliJ IDEA)	Crear los proyectos, agregar dependencias y ejecutar las clases Principal.
Maven	Compilar, empaquetar y ejecutar las pruebas automáticas.
LipeRMI 1.0.1	Librería de terceros que facilita el RMI. Se obtiene desde Maven Repository con las coordenadas <code>com.tascapexq:lipermi:1.0.1</code> , cuyo archivo es <code>lipermi-1.0.1.jar</code> , el mismo que menciona la guía.

```
<dependency>
  <groupId>com.tascapexq</groupId>
  <artifactId>lipermi</artifactId>
  <version>1.0.1</version>
</dependency>
```

Al declarar la dependencia en el archivo `pom.xml`, el IDE o Maven descargan automáticamente el JAR de LipeRMI. Si se trabaja sin Maven, el mismo JAR se agrega a mano en la sección `Libraries` del proyecto, tal como lo muestra la guía.

2.2 Estructura de los tres proyectos

Proyecto	Clases	Responsabilidad
EjemploLipeRmiLibImc	<code>DatosImc</code> , <code>IRemotaCalculoImc</code>	Es la librería compartida: define el objeto que transporta los datos y el contrato remoto que implementan el servidor y usa el cliente.
EjemploLipeRmiServidorImc	<code>CalculoRmiImcImplem</code> , <code>Servidor</code> , <code>Principal</code>	Implementa el contrato remoto, publica el servicio con LipeRMI y contiene el punto de entrada del servidor.
EjemploLipeRmiClienteImc	<code>VentanaPrincipal</code> , <code>Principal</code>	Ofrece el formulario, se conecta al servidor y realiza la invocación remota.

2.3 Paso 1: proyecto tipo librería

2.3.1 Clase para transportar los datos: `DatosImc`

El primer paso de la guía es crear una clase que transporte los datos del IMC. Debe implementar `Serializable`, porque sus objetos se convierten en bytes para viajar por la red entre el cliente y el servidor.

```

package com.jcaa.imc.rmi.lib;

import java.io.Serializable;

public class DatosImc implements Serializable {
    private static final long serialVersionUID = 1L;
    private double peso;
    private double altura;
    private double resultado;
    private String interpretacion;
    // constructores y metodos get/set de cada atributo
}

```

2.3.2 Interfaz para invocar los metodos de forma remota

El segundo paso es definir el contrato remoto. Con LipeRMI es una interfaz Java normal: no hereda de ninguna clase especial ni declara excepciones de red.

```

package com.jcaa.imc.rmi.lib;

public interface IRemotaCalculoImc {
    DatosImc calcularImc(DatosImc datos);
}

```

2.3.3 Construir el archivo distribuible (JAR)

Con Maven, el JAR de la libreria se genera y se instala en el repositorio local con un solo comando; los otros dos proyectos lo consumen como dependencia. En NetBeans equivale a la opcion Clean and Build, que produce el archivo dist/EjemploLipeRmiLibImc.jar que indica la guia.

```

cd EjemploLipeRmiLibImc
mvn clean install

```

2.4 Paso 2: proyecto servidor

2.4.1 Clase CalculoRmiImcImplem

Implementa la interfaz remota. Aqui esta la logica que se ejecuta en el servidor: valida los datos, calcula el IMC y lo clasifica.

```

public class CalculoRmiImcImplem implements IRemotaCalculoImc {

    @Override
    public DatosImc calcularImc(DatosImc datos) {
        if (datos.getPeso() <= 0 || datos.getAltura() <= 0) {
            datos.setResultado(0);
            datos.setInterpretacion(
                "ERROR: El peso y la altura deben ser mayores que 0");
            return datos;
        }
        double resultado = datos.getPeso() / (datos.getAltura() * datos.getAltura());
        datos.setResultado(resultado);
        datos.setInterpretacion(interpretar(resultado));
        return datos;
    }

    private static String interpretar(double imc) {
        if (imc < 18.5) return "Debes consultar un Medico, tu peso es muy bajo";
        else if (imc <= 24.9) return "Estas bien de peso";
        else if (imc <= 29.9) return "Debes bajar un poco de peso";
        else return "Debes consultar un Medico, tu peso es muy alto";
    }
}

```

2.4.2 Clase Servidor: publicar el objeto remoto

La guía crea un CallHandler, un Server y registra el objeto remoto. En cuatro líneas queda publicado el servicio en el puerto 9007.

```
CallHandler invocador = new CallHandler();
Server servidor = new Server();
servidor.bind(puerto, invocador);
invocador.registerGlobal(IRemotaCalculoImc.class, calculoImc);
```

2.4.3 Clase Principal

Es el punto de entrada del servidor: crea el servicio, lo inicia, muestra en consola el puerto y la dirección IP de la máquina (dato que debe escribir el cliente) y queda a la espera. Además se registra un observador que imprime en consola cada vez que un cliente se conecta o se desconecta, lo cual sirve como evidencia de la conexión remota.

2.5 Paso 3: proyecto cliente

2.5.1 Conexión con el servidor

```
CallHandler invocadorRemoto = new CallHandler();
cliente = new Client(ipServidor, puerto, invocadorRemoto);
calculoImcRemoto = (IRemotaCalculoImc) cliente.getGlobal(IRemotaCalculoImc.class);
```

2.5.2 Invocación remota del método

La llamada se escribe igual que una llamada local, pero el método se ejecuta en el servidor. Como la comunicación por red puede tardar, la invocación se hace en un hilo aparte para que la ventana no se congele.

```
DatosImc datos = new DatosImc(peso, altura);
DatosImc respuesta = calculoImcRemoto.calcularImc(datos);
textoResultado.setText("IMC: " + respuesta.getResultado());
textoMensaje.setText(respuesta.getInterpretacion());
```

2.5.3 Ventana del cliente

La ventana contiene los campos de dirección IP, puerto de red, peso y altura; los botones Conectar/Desconectar y Calcular IMC; el área donde se muestra el resultado enviado por el servidor; y un registro con los mensajes enviados y recibidos. La clase Principal lanza esta ventana.

2.6 Paso 4: ejecución y pruebas

```
# Servidor
cd EjemploLipeRmiServidorImc
mvn clean install
java -cp "target/classes;lipermi-1.0.1.jar" com.jcaa.imc.rmi.servidor.Principal

# Cliente (en otra terminal)
cd EjemploLipeRmiClienteImc
mvn clean install
java -cp "target/classes;lipermi-1.0.1.jar" com.jcaa.imc.rmi.cliente.Principal
```

Al ejecutar el servidor se muestra el puerto 9007 y la IP de la máquina. En el cliente se escribe esa IP, se presiona Conectar (en la consola del servidor aparece el mensaje CLIENTE CONECTADO con la IP y el puerto del cliente) y luego se ingresan el peso y la altura para obtener el resultado.

2.6.1 Resultados verificados

Peso (kg)	Altura (m)	Respuesta del servidor
70	1.75	IMC 22.86 - Estas bien de peso
45	1.75	Debes consultar un Medico, tu peso es muy bajo
85	1.75	Debes bajar un poco de peso
120	1.75	Debes consultar un Medico, tu peso es muy alto
0	1.75	ERROR: El peso y la altura deben ser mayores que 0

2.6.2 Pruebas automaticas

El proyecto servidor incluye cuatro pruebas que se ejecutan con el comando mvn install: tres comprueban el calculo y las categorias del IMC, y una prueba de integracion levanta el servidor LipeRMI en un puerto libre, se conecta con un cliente LipeRMI real y verifica la respuesta, comprobando asi que la invocacion remota funciona de extremo a extremo.

3. Punto 2: el mismo ejemplo con RMI estandar de Java

En este punto se repite el ejemplo usando unicamente el paquete `java.rmi` incluido en el JDK. La estructura sigue siendo de tres proyectos -libreria, servidor y cliente- y el resultado que ve el usuario es exactamente el mismo; lo que cambia es la forma de declarar el contrato, de publicar el objeto remoto y de conectarse desde el cliente.

3.1 Paso 1: proyecto libreria

La clase `DatosImc` se mantiene igual, porque en las dos tecnologias los objetos que viajan por la red deben implementar `Serializable`. La interfaz, en cambio, obedece ahora las reglas del paquete `java.rmi`.

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface IRemotaCalculoImc extends Remote {
    String NOMBRE_SERVICIO = "CalculoImc";
    DatosImc calcularImc(DatosImc datos) throws RemoteException;
}
```

Diferencias obligatorias de esta tecnologia: la interfaz debe extender `java.rmi.Remote` y cada metodo remoto debe declarar `throws RemoteException`. Con `LipeRMI` ninguna de las dos cosas es necesaria.

3.2 Paso 2: proyecto servidor

La clase que implementa el servicio debe extender `UnicastRemoteObject`, porque es la clase del JDK que exporta el objeto para que pueda recibir llamadas por la red. El constructor y el metodo remoto lanzan `RemoteException`.

```
public class CalculoRmiImcImplem extends UnicastRemoteObject
    implements IRemotaCalculoImc {

    public CalculoRmiImcImplem() throws RemoteException { super(); }

    public CalculoRmiImcImplem(int puerto) throws RemoteException { super(puerto); }

    @Override
    public DatosImc calcularImc(DatosImc datos) throws RemoteException {
        // la misma logica de calculo y validacion del punto 1
    }
}
```

Para publicar el servicio se crea el registro RMI y se asocia un nombre al objeto remoto. La guia usa un `CallHandler` y un `Server`; aqui se usan las clases `LocateRegistry` y `Naming` del propio JDK.

```
implementacion = new CalculoRmiImcImplem(puertoObjeto);
registro = LocateRegistry.createRegistry(puertoRegistro);
registro.rebind(IRemotaCalculoImc.NOMBRE_SERVICIO, implementacion);
```

En el punto de entrada del servidor se publica ademas la propiedad `java.rmi.server.hostname` con la IP de la maquina, para que el objeto remoto que recibe el cliente apunte a la direccion correcta cuando el cliente esta en otro equipo. Con RMI estandar se usan dos puertos: el 1099 para el registro `rmiregistry` y el 1100 para el objeto remoto exportado.

3.3 Paso 3: proyecto cliente

El cliente no necesita el JAR de ninguna libreria externa: solo la interfaz remota y la clase de datos. La conexion se hace buscando el nombre del servicio en el registro.

```
String url = "rmi://" + host + ":" + puertoRegistro + "/"
        + IRemotaCalculoImc.NOMBRE_SERVICIO;
calculoImcRemoto = (IRemotaCalculoImc) Naming.lookup(url);
```

La invocacion del metodo remoto es identica a la del punto 1, con la diferencia de que el compilador obliga a manejar RemoteException:

```
DatosImc respuesta = calculoImcRemoto.calcularImc(new DatosImc(peso, altura));
```

3.4 Paso 4: ejecucion y pruebas

```
# Servidor (crea el registro en 1099 y exporta el objeto en 1100)
cd EjemploRmiEstandarServidorImc
mvn clean install
java -cp target/classes com.jcaa.imc.rmi.servidor.Principal

# Cliente
cd EjemploRmiEstandarClienteImc
mvn clean install
java -cp target/classes com.jcaa.imc.rmi.cliente.Principal
```

El registro RMI se puede crear dentro del programa, como se hace en esta entrega, o ejecutando aparte el comando rmiregistry 1099 que viene con el JDK. Los resultados del IMC obtenidos son los mismos de la tabla de la seccion 2.6.1 y las cuatro pruebas automaticas tambien pasan, incluida una prueba de integracion que crea el registro, publica el objeto y lo invoca con Naming.lookup, recorriendo el mismo camino que usa el cliente real.

3.5 Diferencias y similitudes entre RMI estandar y LipeRMI

Aspecto	RMI estandar (java.rmi)	LipeRMI
Libreria	Incluida en el JDK	Libreria de terceros: lipermi-1.0.1.jar
Interfaz remota	Debe extender java.rmi.Remote	Interfaz Java normal
Excepciones	Cada metodo declara throws RemoteException	No declara excepciones de red
Objeto remoto	Debe extender UnicastRemoteObject o exportarse con exportObject	Clase normal registrada con registerGlobal
Registro	Necesita rmiregistry en el puerto 1099 mas un puerto para el objeto	No necesita registro; un solo puerto TCP
Conexion del cliente	Naming.lookup("rmi://host:puerto/nombre")	new Client(host, puerto, handler) y getGlobal(...)
Serializacion	Java serialization (clases Serializable)	Java serialization (igual)
Transporte	JRMP sobre TCP	Protocolo propio de la libreria sobre TCP
Puertos en el firewall	Normalmente dos o mas	Uno solo
Publicar el servicio	Crear registro, exportar y rebind	bind y registerGlobal
Invocar el metodo	stub.calcularImc(datos)	proxy.calcularImc(datos)
Dependencia extra	Ninguna	El JAR de LipeRMI en cliente y servidor
Ventaja principal	Es el estandar del lenguaje y no depende de terceros	Menos codigo y menos configuracion de red

3.5.1 Similitudes

- En las dos tecnologías se define una interfaz con los metodos que se pueden invocar de forma remota, y el servidor entrega una implementacion de esa interfaz.
- Se necesita una clase de datos que implemente Serializable para transportar la informacion entre cliente y servidor.
- El cliente se conecta por red a un servicio que esta publicado con un nombre y luego invoca el metodo igual que si fuera local; el programador no escribe sockets.
- En los dos casos la logica de negocio se ejecuta en el servidor y el cliente solo recibe el resultado.

3.5.2 Conclusion del punto

RMI estandar es la opcion natural cuando todos los clientes son Java y se quiere evitar dependencias externas, a costa de escribir mas codigo y de configurar el registro y los puertos. LipeRMI simplifica el desarrollo y la configuracion de red, y es comodo en ejemplos academicos o cuando no se pueden abrir varios puertos, aunque obliga a distribuir su JAR con la aplicacion.

4. Punto 3: otras librerías para RMI/RPC

Se revisaron librerías de Java y de otros lenguajes que permiten construir sistemas distribuidos con invocación remota, teniendo en cuenta el lenguaje, el transporte que usan, la forma de describir el contrato y el tipo de serialización.

4.1 Librerías encontradas

Librería	Lenguaje	Transporte	Contrato	Serialización
RMI estandar (java.rmi)	Java	TCP (JRMP)	Interfaz que extiende Remote	Java serialization
LipeRMI	Java y Android	TCP propio	Interfaz Java normal	Java serialization
gRPC	Java, C#, Go, Python, Node	HTTP/2	Archivo .proto	Protocol Buffers
Apache Thrift	Múltiples lenguajes	TCP o HTTP	Archivo .thrift	Binaria propia
Apache Dubbo	Java	TCP o HTTP	Interfaz Java	Hessian2, JSON o Protobuf
Spring Remoting	Java	RMI, HTTP, JMS	Interfaz Java	Según el protocolo
JSON-RPC (jsonrpc4j, jayson)	Java, Node, Python	HTTP	Métodos con nombre	JSON (texto)
Cap'n Proto RPC	C++, Java, Python	TCP	Archivo .capnp	Binaria muy rápida
Pyro5	Python	TCP	Objetos Python	Pickle o serpent
RPyC	Python	TCP	Objetos Python	Propia
net/rpc (biblioteca estandar)	Go	TCP o HTTP	Métodos exportados	gob
jayson / grpc-js	Node.js	TCP o HTTP/2	JSON-RPC o .proto	JSON o Protobuf
.NET Remoting o gRPC	C#	TCP o HTTP/2	Interfaz o .proto	Binaria

4.2 Análisis de la información

- Todas las librerías siguen la misma idea: se describe el servicio en un contrato (una interfaz o un lenguaje de descripción) y el cliente obtiene un objeto sustituto, llamado stub o proxy, con el que llama al método como si fuera local.
- La diferencia principal está en cómo se describen los datos. RMI y LipeRMI serializan objetos Java, mientras que gRPC y Thrift usan un archivo de descripción que permite generar código para varios lenguajes de programación.
- RMI y LipeRMI son cómodos cuando cliente y servidor son Java; gRPC y Thrift son la opción cuando conviven varios lenguajes o cuando se necesita HTTP/2 y alto rendimiento.
- Para prototipos rápidos en otros lenguajes existen opciones muy sencillas, como Pyro5 en Python o net/rpc en Go, que permiten tener un ejemplo funcional con pocas líneas.
- En entornos empresariales actuales, gRPC es la librería de RPC más utilizada, seguida de Thrift y de las soluciones propias de cada plataforma, como Dubbo en el ecosistema Java.

4.3 Ejemplo muy simple desarrollado con gRPC

Para cumplir el tercer punto se desarrollo un ejemplo minimo y funcional con gRPC, que es la libreria de RPC mas usada actualmente y ademas es multilenguaje. El ejemplo calcula el mismo IMC del resto de la actividad.

4.3.1 El contrato: archivo *imc.proto*

En gRPC el contrato se escribe en un lenguaje de descripcion de interfaces. A partir de ese archivo, la herramienta protoc genera las clases de los mensajes y los stubs del cliente y del servidor.

```
syntax = "proto3";
package imc;
option java_package = "com.jcaa.imc.grpc";

message DatosImc {
    double peso = 1;
    double altura = 2;
}

message ResultadoImc {
    double imc = 1;
    string interpretacion = 2;
}

service CalculoImc {
    rpc Calcular (DatosImc) returns (ResultadoImc);
}
```

4.3.2 Servidor

```
public class ServidorGrpc {
    public static void main(String[] args) throws Exception {
        Server servidor = ServerBuilder.forPort(50051)
            .addService(new CalculoImcService())
            .build()
            .start();
        servidor.awaitTermination();
    }

    static class CalculoImcService extends CalculoImcGrpc.CalculoImcImplBase {
        @Override
        public void calcular(DatosImc petition,
            StreamObserver<ResultadoImc> observador) {
            double imc = petition.getPeso()
                / (petition.getAltura() * petition.getAltura());
            observador.onNext(ResultadoImc.newBuilder()
                .setImc(imc).setInterpretacion(interpretar(imc)).build());
            observador.onCompleted();
        }
    }
}
```

4.3.3 Cliente

```
ManagedChannel canal = ManagedChannelBuilder
    .forAddress("localhost", 50051)
    .usePlaintext()
    .build();

CalculoImcGrpc.CalculoImcBlockingStub stub = CalculoImcGrpc.newBlockingStub(canal);
ResultadoImc respuesta = stub.calcular(DatosImc.newBuilder()
    .setPeso(70).setAltura(1.75).build());
System.out.println("IMC: " + respuesta.getImc());
canal.shutdown();
```

4.3.4 Ejecucion y resultado obtenido

```
cd ejemplo-grpc-java
mvn clean package          # descarga protoc y genera el codigo

java -cp "target/classes;target/cp.txt" com.jcaa.imc.grpc.ServidorGrpc
java -cp "target/classes;target/cp.txt" com.jcaa.imc.grpc.ClienteGrpc 70 1.75
```

Salida obtenida al ejecutar el cliente:

```
Peso: 70.00 kg - Altura: 1.75 m
IMC: 22.86
Interpretacion: Estas bien de peso
```

Con un peso invalido (0 kg) el servidor responde IMC 0.00 y el mensaje "ERROR: El peso y la altura deben ser mayores que 0", igual que en los puntos anteriores. Esto demuestra que el mismo problema se puede resolver con librerias distintas y que en todos los casos el contrato y los datos son el centro del diseno.

5. Resultados y verificación

Todo el material se compilo y se ejecuto antes de entregarlo. La tabla resume el estado de cada punto.

Punto	Compilacion	Pruebas automaticas	Verificacion manual
1 - LipeRMI	Los 3 proyectos compilan y generan su JAR	4 pruebas correctas: 3 de calculo y categorias, 1 de integracion con servidor y cliente LipeRMI reales	Servidor y cliente ejecutados por separado; el resultado del IMC se calculo en el servidor
2 - RMI estandar	Los 3 proyectos compilan y generan su JAR	4 pruebas correctas: 3 de calculo y categorias, 1 de integracion con registro RMI y Naming.lookup	Servidor con registro en el puerto 1099 y objeto en el 1100; cliente conectado y resultado correcto
3 - gRPC	El proyecto compila; protoc genera las clases y los stubs	Se verifico la comunicacion ejecutando servidor y cliente	IMC 22.86 con datos validos y mensaje de error con peso igual a cero

5.1 Casos de prueba utilizados en las dos tecnologias RMI

Caso	Peso (kg)	Altura (m)	Resultado esperado
Peso normal	70	1.75	IMC 22.86 - Estas bien de peso
Bajo peso	45	1.75	Debes consultar un Medico, tu peso es muy bajo
Sobrepeso	85	1.75	Debes bajar un poco de peso
Obesidad	120	1.75	Debes consultar un Medico, tu peso es muy alto
Dato invalido	0	1.75	ERROR: El peso y la altura deben ser mayores que 0
Dato invalido	70	-1	ERROR: El peso y la altura deben ser mayores que 0

5.2 Como reproducir la verificacion

```
# Punto 1
cd 1-lipermi && mvn clean install

# Punto 2
cd 2-rmi-estandar && mvn clean install

# Punto 3
cd 3-investigacion/ejemplo-grpc-java && mvn clean package
```

6. Conclusiones

- Se completo el ejemplo de la guía (calculadora de IMC) con la librería LipeRMI, entregando los tres proyectos que pide la actividad: la librería compartida, el servidor y el cliente. La lógica de cálculo se ejecuta en el servidor y el cliente solo envía los datos y recibe el resultado.
- Se repitió el mismo ejemplo con RMI estándar de Java. Aunque el resultado para el usuario es idéntico, el contrato cambia: la interfaz debe extender `java.rmi.Remote`, los métodos declaran `RemoteException` y el objeto remoto debe extender `UnicastRemoteObject`; además se requiere el registro `rmiregistry` y, en general, dos puertos.
- LipeRMI reduce el código y la configuración porque no exige herencia ni excepciones declaradas y publica el servicio en un solo puerto. RMI estándar no necesita ninguna librería adicional y es el estándar del lenguaje, pero da más trabajo de configuración.
- En cuanto a otras alternativas, gRPC es hoy la librería de RPC más usada y destaca por ser multilenguaje, usar HTTP/2 y generar automáticamente el código a partir de un contrato; Apache Thrift ofrece una idea similar, mientras que Pyro5, RPyC o net/rpc son opciones muy sencillas en otros lenguajes.
- El aprendizaje principal es que la tecnología cambia la forma de transportar la llamada, pero el diseño se mantiene: un contrato bien definido, datos serializables y la lógica de negocio en el servidor.

7. Referencias

- Arrieta Arrieta, J. C. Guia de desarrollo de aplicaciones distribuidas usando RMI. Documento de clase.
- Oracle. Java Remote Method Invocation (RMI) - documentacion oficial del paquete java.rmi. <https://docs.oracle.com/en/java/javase/17/docs/api/java.rmi/java/rmi/package-summary.html>
- The LipeRMI Project. LipeRMI - libreria de invocacion remota de metodos para Java. <http://lipermi.sourceforge.net/>
- Maven Repository. LipeRMI 1.0.1 (com.tascape.qa:lipermi:1.0.1). <https://mvnrepository.com/artifact/com.tascape.qa:lipermi>
- gRPC Authors. Introduction to gRPC y documentacion de Java. <https://grpc.io/docs/>
- Apache Software Foundation. Apache Thrift. <https://thrift.apache.org/>
- Apache Software Foundation. Apache Dubbo. <https://dubbo.apache.org/>
- Irmen de Jong. Pyro5 - Python Remote Objects. <https://pyro5.readthedocs.io/>
- The Go Authors. Package net/rpc. <https://pkg.go.dev/net/rpc>
- Apache Software Foundation. Apache Maven - documentacion de referencia. <https://maven.apache.org/>